

Fiche 302 — Indexation : les quatre index, `[`, `[[`, `$` et l'assignation par sous-ensemble

Matière	Code · Langage R
Cours source	R Core Team, <i>An Introduction to R</i> 4.6.1 — §2.7 « Index vectors; selecting and modifying subsets of a data set »
Sources d'appoint	<i>R Language Definition</i> 4.6.1, §3.4 « Indexing » — §3.4.1 indexation par vecteurs, §3.4.2 matrices et tableaux, §3.4.3 autres structures, §3.4.4 assignation par sous-ensemble
Difficulté	Fondamental — l'opération la plus fréquente de R, et celle qui a le plus de cas particuliers
Temps d'étude estimé	1 h 30
Prérequis	Fiches 300 et 301 (recyclage, <code>NA</code> , vecteurs logiques)
Concepts clés	les quatre types d'index (logique, entier positif, entier négatif, caractère), <code>[</code> contre <code>[[</code> contre <code>\$</code> , dépassement de bornes, index zéro, troncature, recyclage d'un index logique, appariement partiel et argument <code>exact</code> , index facteur, <code>x[]</code> , <code>NULL</code> , index <code>NA</code> , index matriciel, <code>drop = FALSE</code> , tableau à une dimension, <code>[.data.frame</code> , environnements, assignation par sous-ensemble et le mécanisme <code>`*tmp*`</code> , fonctions de remplacement
À retenir en priorité	Les quatre types d'index · <code>[</code> garde la structure, <code>[[</code> extrait l'élément · l'appariement partiel de <code>\$</code> et <code>[[</code> · <code>drop = FALSE</code> · le fait que <code>x[i] <- v</code> est un appel de fonction.

Vue d'ensemble

TROIS OPERATEURS	<code>x[i]</code>	conserve la structure ; accepte un VECTEUR d'indices
	<code>x[[i]]</code>	extrait UN element ; perd noms et dimnames
	<code>x\$a</code>	un NOM LITTERAL seulement -- l'index n'est pas calculable

QUATRE INDEX (R-intro 2.7)

logique	<code>x[!is.na(x)]</code>	recycle si trop court, NA si trop long
entier POSITIF	<code>x[1:10]</code>	hors bornes -> NA
entier NEGATIF	<code>x[-(1:5)]</code>	= « tout sauf » ; hors bornes -> IGNORE
caractere	<code>fruit["apple"]</code>	via l'attribut names

```
... plus quatre cas de bord (R-lang 3.4.1)
facteur = x[as.integer(i)] -- les NIVEAUX NE SONT PAS UTILISES
vide    x[]  rend x en laissant tomber les attributs « non pertinents »
NULL    traite comme integer(0)
NA       rend NA. x[NA] a la longueur de x ; x[c(1,NA)] a la longueur 2
```

SIGNES tous les elements d'un index entier ont LE MEME SIGNE
ZERO x[0] est vide ; un zero melange aux autres est IGNORE

MATRICES m[i] un seul indice -> c(m)[i], dim ignore
 m[i, j] deux indices
 m[idx] idx MATRICE d'entiers : une ligne = un element
 UNE DIMENSION DE LONGUEUR 1 EST SUPPRIMEE -> drop = FALSE

ASSIGNER x[3:5] <- 13:15 est un APPEL DE FONCTION :
 x <- `[<-`(x, 3:5, value = 13:15)

Le problème posé. « *Subsets of the elements of a vector may be selected by **appending to the name of the vector an index vector in square brackets*** » (§2.7). L'idée est simple ; ce qui ne l'est pas, c'est que **le crochet accepte sept sortes d'index différentes**, dont trois se comportent de façon contre-intuitive. C'est là que se logent la moitié des bugs silencieux d'un débutant.

INTUITION — INDEXER, C'EST APPELER UNE FONCTION.

« *Like the other operators, **indexing is really done by functions**, and one could have used ``[`(x, 2)` instead of `x[2]`* » (R Language Definition §3.1.4). Le crochet n'est pas une syntaxe spéciale du langage : c'est **une fonction générique** dont on peut écrire des méthodes (§3.4.3), qu'on peut passer en argument, et dont il existe une version **de remplacement** (``[<-``) qui explique tout le concept 8.

● Concept 1 — Trois opérateurs, trois intentions

DÉFINITION (R LANGUAGE DEFINITION §3.4).

« R has **three basic indexing operators**, with syntax displayed by the following examples : »

```

x[i]
x[i, j]
x[[i]]
x[[i, j]]
x$a
x$"a"

```

	[	[[	\$
Intention	prendre un sous-ensemble	extraire un élément	extraire un élément nommé
Nombre d'éléments	un vecteur d'indices	un seul	un seul
Type du résultat	le même que l'objet	le type de l'élément	le type de l'élément
Index calculable	oui	oui	non
Appariement partiel	jamais	oui (contrôlable)	oui

1.1 Ce que [[fait de plus que [

Règle (§3.4). « For vectors and matrices **the [[forms are rarely used**, although they have some **slight semantic differences** from the [form : e.g. **it drops any names or dimnames attribute**, and that **partial matching is used for character indices**. »

« When indexing multi-dimensional structures with a single index, `x[[i]]` or `x[i]` will return **the i-th sequential element** of `x`. »

« For lists, one generally uses `[[` to select any single element, whereas `[` returns a list of the selected elements. »

C'est **la** distinction à retenir pour les listes (fiche 306) :

```

l <- list(a = 1:3, b = "texte")
l[1]      # une LISTE de un element
l[[1]]    # le VECTEUR 1:3 lui-meme

```

Règle — pourquoi `[]` n'accepte qu'un élément (§3.4). « The `[]` form allows **only a single element to be selected** using integer or character indices, whereas `[]` allows indexing by vectors. »

« Note though that **for a list or other recursive object, the index can be a vector and each element of the vector is applied in turn** to the list, the selected component, the selected component of that component, and so on. **The result is still a single element.** »

Autrement dit `1[[c(1, 2)]]` **n'extrait pas deux éléments** : il extrait le 1^{er} élément de `1`, puis le 2^e élément **de celui-ci**. C'est une **descente en profondeur**, pas une sélection multiple.

1.2 Ce que `$` a de particulier

Règle (§3.4). « The form using `$` applies to **recursive objects** such as lists and pairlists. It allows **only a literal character string or a symbol** as the index. That is, **the index is not computable** : for cases where you need to **evaluate an expression** to find the index, use `x[[expr]]`. **Applying `$` to a non-recursive object is an error.** »

C'est le piège n° 1 de la programmation en R.

```
colonne <- "prix"
donnees$colonne      # cherche une colonne littéralement nommée « colonne » -> NULL
donnees[[colonne]]   # cherche la colonne « prix »                <- CE QU'ON VOULAIT
```

Le premier ne lève **aucune erreur** : il rend `NULL`. Le symptôme apparaît plus loin, sous une forme incompréhensible. **Dès qu'un nom de colonne est dans une variable, `$` est le mauvais outil.**

● Concept 2 — Les quatre types d'index du cours

DÉFINITION (§2.7).

« More generally, **any expression that evaluates to a vector** may have subsets of its elements similarly selected by appending an index vector in square brackets immediately after the expression. **Such index vectors can be any of four distinct types.** »

2.1 Type 1 — un vecteur logique

« In this case the index vector is **recycled to the same length** as the vector from which elements are to be selected. **Values corresponding to `TRUE` in the index vector are selected and those corresponding to `FALSE` are omitted.** » (§2.7)

```
y <- x[!is.na(x)]
```

« creates (or re-creates) an object `y` which will contain **the non-missing values of `x`**, in the same order. **Note that if `x` has missing values, `y` will be shorter than `x`.** »

C'est le détail qui casse les analyses. Filtrer raccourcit. Si vous filtrez `x` mais pas `y`, les deux vecteurs ne sont plus alignés — et R ne dira rien, il recyclera (fiche 301).

L'exemple composé du cours, à lire de l'intérieur vers l'extérieur :

```
(x+1)[(!is.na(x)) & x > 0] -> z
```

« creates an object `z` and places in it the values of the vector `x+1` for which the corresponding value **in `x`** was both **non-missing and positive.** »

Trois choses y sont remarquables : l'objet indexé est **une expression** `(x+1)`, pas un nom ; la condition porte sur `x` et non sur `x+1` ; et les deux conditions sont combinées par `&`, l'opérateur vectorisé (fiche 301).

COMPLÉMENT — LES DEUX LONGUEURS ANORMALES (R LANGUAGE DEFINITION §3.4.1).

*« The indexing `i` should **generally have the same length as** `x` . If it is shorter, then its elements will be recycled ; if it is longer, then `x` is conceptually extended with `NA` s. »*

Longueur de l'index logique	Comportement
égale	le cas normal
plus courte	recyclée — d'où l'idiome <code>x[c(TRUE, FALSE)]</code> qui prend un élément sur deux
plus longue	<code>x</code> est prolongé par des <code>NA</code> — on obtient des <code>NA</code> en fin de résultat

2.2 Type 2 — des entiers positifs

*« In this case the values in the index vector **must lie in the set** `{1, 2, ..., length(x)}`. The corresponding elements are selected and concatenated, **in that order**, in the result. **The index vector can be of any length and the result is of the same length as the index vector.** »* (§2.7)

```
x[6]      # le sixieme element
x[1:10]    # les dix premiers (si length(x) >= 10)
```

L'exemple délibérément étrange du cours, qui montre que **l'index peut être plus long que le vecteur** :

```
c("x","y")[rep(c(1,2,2,1), times = 4)]
```

« (an admittedly unlikely thing to do) produces a character vector **of length 16** consisting of `"x"`, `"y"`, `"y"`, `"x"` **repeated four times.** » Un vecteur de **2** éléments indexé par **16** indices donne **16** éléments : **la longueur du résultat est celle de l'index**, pas celle du vecteur.

COMPLÉMENT — LE HORS-BORNES ET LE ZÉRO (R LANGUAGE DEFINITION §3.4.1).

- « *If `i` is positive and exceeds `Length(x)` then the corresponding selection is `NA`.* »
- *« **Negative out of bounds values for `i` are silently disregarded** since R version **2.6.0**, S compatibly, as they mean to **drop non-existing elements** and that is **an empty operation ("no-op")**. »*
- « *A special case is the zero index, which has null effects : `x[0]` is an empty vector and otherwise including zeros among positive or negative indices has the same effect as if they were omitted.* »
- « *All elements of `i` must have the same sign.* »

L'asymétrie positif / négatif est un piège de diagnostic. Un indice positif hors bornes **produit un `NA`** — un symptôme visible. Un indice négatif hors bornes est **silencieusement ignoré**. Sur `x` de longueur 5 : `x[7]` rend `NA` , `x[-7]` rend `x` **en entier**, sans le moindre signe.

COMPLÉMENT — LES INDEX NON ENTIERS.

« **Other numeric.** Non-integer values are **converted to integer (by truncation towards zero)** before use. » Donc `x[2.9]` rend `x[2]` , sans avertissement. Une division qui aurait dû être `%%` passe inaperçue.

2.3 Type 3 — des entiers négatifs

« *Such an index vector specifies the values **to be excluded rather than included**.* » (§2.7)

```
y <- x[-(1:5)]      # « tout sauf les cinq premiers »
```

Les parenthèses sont obligatoires. `x[-1:5]` n'est pas `x[-(1:5)]` : le `:` étant prioritaire (fiche 301), `-1:5` vaut `c(-1, 0, 1, 2, 3, 4, 5)` — des signes **mélangés**, ce que la règle interdit : « *All elements of `i` must have the same sign* ». R lève alors une erreur — heureusement.

2.4 Type 4 — des chaînes de caractères

« This possibility **only applies where an object has a `names` attribute** to identify its components. In this case a sub-vector of the `names` vector may be used in the same way as the positive integral labels. » (§2.7)

```
fruit <- c(5, 10, 1, 20)
names(fruit) <- c("orange", "banana", "apple", "peach")
lunch <- fruit[c("apple", "orange")]      # 1 et 5, DANS CET ORDRE
```

« The advantage is that **alphanumeric names are often easier to remember than numeric indices**. This option is particularly useful **in connection with data frames**. »

Et l'ordre suit l'index, pas l'objet : `fruit[c("apple", "orange")]` rend `apple` **d'abord**. C'est la même règle qu'au type 2 — « *concatenated, in that order* ».

COMPLÉMENT DÉCISIF — L'APPARIEMENT PARTIEL (R LANGUAGE DEFINITION §3.4.1).

*« The strings in `i` are matched against the `names` attribute of `x` and the resulting integers are used. For `[]` and `$` **partial matching is used if exact matching fails**, so `x$aa` will match `x$aabb` **if `x` does not contain a component named "aa" and "aabb" is the only name which has prefix "aa"**. »*

*« For `[]`, partial matching can be controlled via the `exact` argument which **defaults to `NA`**, indicating that partial matching **is allowed but should result in a warning** when it occurs. Setting `exact` to `TRUE` **prevents** partial matching, a `FALSE` value **allows it and does not issue any warnings**. Note that `[]` **always requires an exact match**. »*

Opérateur	Appariement partiel
<code>[]</code>	jamais
<code>[]</code>	<code>exact = NA</code> (défaut) : autorisé avec avertissement · <code>TRUE</code> : interdit · <code>FALSE</code> : autorisé en silence
<code>\$</code>	oui, silencieusement

« Note that **partial matching is only used when extracting and not when replacing**. »

Conséquence redoutable : `1$val` **lit** `1$va1eur` par appariement partiel, mais `1$val <- 3` **crée un nouvel élément** `va1` . On croit modifier ; on ajoute.

*« The string `""` is treated specially : it indicates **"no name"** and **matches no element (not even those without a name)**. »*

● Concept 3 — Les quatre cas de bord

Ces quatre-là ne sont pas dans *An Introduction to R*. Ils sont dans la *R Language Definition* §3.4.1, et chacun explique un comportement qui paraît autrement absurde.

3.1 Un facteur comme index

PIÈGE (§3.4.1).

*« **Factor.** The result is identical to `x[as.integer(i)]` . **The factor levels are never used.** If so desired, use `x[as.character(i)]` or a similar construction. »*

C'est le piège le plus vicieux de la section, parce qu'un facteur **s'affiche** avec ses étiquettes tout en **valant** ses codes entiers (fiche 304). Indexer par un facteur indexe donc **par des positions 1, 2, 3...** que rien à l'écran ne laisse deviner. Le résultat a la bonne longueur, il est simplement faux.

3.2 L'index vide `x[]`

Règle (§3.4.1). *« **Empty.** The expression `x[]` returns `x` , but **drops "irrelevant" attributes** from the result. **Only** `names` , and in multi-dimensional arrays `dim` and `dimnames` attributes, are retained. »*

`x[]` n'est donc pas une opération neutre : c'est **un nettoyage d'attributs**. Et surtout, à gauche d'une assignation, `x[] <- 0` **conserve la forme de** `x` alors que `x <- 0` la détruit.

3.3 `NULL`

*« **NULL.** This is treated as if it were `integer(0)` . »* — donc `x[NULL]` rend un vecteur **vide**, comme `x[0]` .

3.4 Un index NA — et la longueur qui change de règle

PROBLÈME (§3.4.1).

*« Indexing with a **missing (i.e. NA) value gives an NA result**. This rule applies **also to the case of logical indexing** : the elements of `x` that have an `NA` selector in `i` **get included in the result, but their value will be NA** . »*

*« Notice however that there are **different modes of NA** — the literal constant is of mode "logical" , but it is frequently automatically coerced to other types. One effect of this is that `x[NA]` **has the length of `x`** , but `x[c(1, NA)]` **has length 2**. That is because **the rules for logical indices apply in the former case, but those for integer indices in the latter**. »*

Écriture	Type effectif de l'index	Longueur du résultat
<code>x[NA]</code>	logique (le type par défaut de <code>NA</code>) — recyclé sur toute la longueur	<code>length(x)</code> , tout en <code>NA</code>
<code>x[c(1, NA)]</code>	entier (<code>c(1, NA)</code> est coercé en entier)	2

Une même constante `NA` , deux comportements, parce que **son type a changé au moment de la concaténation**. C'est la conséquence directe de la règle vue en fiche 301 : « *the default type of NA is logical, unless coerced to some other type* ».

Une dernière règle, simple mais utile : « **Indexing with `[]` will also carry out the relevant subsetting of any names attributes**. » Les noms suivent le sous-ensemble.

Énoncé. Soit `x <- c(a = 10, b = 20, c = 30, d = 40, e = 50)` . Donner longueur et contenu de :
`x[2.9]` · `x[7]` · `x[-7]` · `x[0]` · `x[c(1, 0, 3)]` · `x[NA]` · `x[c(1, NA)]` .

Étape 1 — `x[2.9]` . Règle « Other numeric » : « *converted to integer by truncation towards zero* ».
`2.9` devient `2` . Résultat : `b = 20` , longueur **1**. Aucun avertissement.

Étape 2 — `x[7]` . Entier positif dépassant `length(x) = 5` . « *If `i` is positive and exceeds `length(x)` then the corresponding selection is NA* » . Résultat : `NA` , longueur **1**, avec un nom `<NA>` .

Étape 3 — `x[-7]` . Entier négatif hors bornes. « **Negative out of bounds values for `i` are silently disregarded** » depuis R 2.6.0 — retirer un élément inexistant est une opération vide. Résultat : `x`

en entier, longueur **5**. Asymétrie complète avec l'étape 2 : le positif signale, le négatif se tait.

Étape 4 — `x[0]` . « `x[0]` is **an empty vector** ». Longueur **0**, du même type que `x` .

Étape 5 — `x[c(1, 0, 3)]` . « *including zeros among positive or negative indices has the same effect as if they were omitted* ». L'index vaut donc `c(1, 3)` . Résultat : `a = 10`, `c = 30` , longueur **2** — pas 3.

Étape 6 — `x[NA]` . Le `NA` nu est **de mode logique**. Les règles logiques s'appliquent : index recyclé sur toute la longueur de `x` , et chaque sélecteur `NA` fait entrer l'élément **avec la valeur** `NA` . Longueur **5**, cinq `NA` .

Étape 7 — `x[c(1, NA)]` . `c(1, NA)` force le `NA` **en entier**. Les règles entières s'appliquent : deux indices, donc **longueur 2** — `a = 10` puis `NA` .

Étape 8 — *la leçon*. Quatre de ces sept expressions rendent quelque chose **sans le moindre message**, dont deux d'une longueur inattendue. En R, **la vérification à faire après un sous-ensemble est** `length()` , systématiquement.

● Concept 4 — Indexer une matrice ou un tableau

Règle générale (§3.4.2). *« Subsetting multi-dimensional structures **generally follows the same rules as single-dimensional indexing for each index variable**, with the relevant component of `dimnames` taking the place of `names` . **A couple of special rules apply, though.** »*

4.1 Un seul indice sur une matrice

*« Normally, a structure is accessed using **the number of indices corresponding to its dimension**. It is however also possible to use **a single index**, in which case **the `dim` and `dimnames` attributes are disregarded** and the result is effectively that of `c(m)[i]` . **Notice that `m[1]` is usually very different from `m[1, ]` or `m[, 1]` .** »*

Écriture	Ce que R rend
<code>m[1]</code>	le premier élément du vecteur sous-jacent, <code>dim</code> ignoré
<code>m[1,]</code>	la première ligne
<code>m[, 1]</code>	la première colonne

C'est la conséquence directe de ce qu'annonçait le §2.8 : une matrice **est** un vecteur (fiches 301 et 303). Le `dim` est un attribut ; avec un seul indice, il est simplement **mis de côté**.

4.2 Une matrice d'entiers comme index

MÉTHODE (§3.4.2).

« *It is possible to use **a matrix of integers as an index**. In this case, **the number of columns of the matrix should match the number of dimensions of the structure**, and the result will be **a vector with length as the number of rows of the matrix**.* »

L'exemple du manuel, qui extrait `m[1, 1]` et `m[2, 2]` **en une seule opération** :

```
m <- matrix(1:4, 2)
m
#      [,1] [,2]
# [1,]    1    3
# [2,]    2    4

i <- matrix(c(1, 1, 2, 2), 2, byrow = TRUE)
i
#      [,1] [,2]
# [1,]    1    1
# [2,]    2    2

m[i]
# [1] 1 4
```

Comment le lire : chaque **ligne** de `i` est un **jeu de coordonnées**. Ligne 1 → `(1,1)` → la valeur **1**. Ligne 2 → `(2,2)` → la valeur **4**. Deux lignes dans l'index, deux valeurs en sortie. C'est ainsi qu'on extrait la

diagonale, ou n'importe quel ensemble de cases dispersées, sans boucle.

Les restrictions (§3.4.2). « *Indexing matrices may not contain negative indices. NA and zero values are allowed : rows in an index matrix containing a zero are ignored, whereas rows containing an NA produce an NA in the result.* »

Et : « *Both in the case of using a single index and in matrix indexing, a names attribute is used if present, as had the structure been one-dimensional.* »

4.3 drop — la dimension qui disparaît

Le piège que le manuel qualifie lui-même de « cause fréquente d'échec » (§3.4.2). « *If an indexing operation causes the result to have one of its extents of length one — as in selecting a single slice of a three-dimensional matrix with (say) `m[2, , ]` — the corresponding dimension is generally dropped from the result. If a single-dimensional structure results, a vector is obtained.* »

« *This is occasionally undesirable and can be turned off by adding `drop = FALSE` to the indexing operation. Notice that this is an additional argument to the `[` function and doesn't add to the index count. Hence the correct way of selecting the first row of a matrix as a 1 par n matrice is `m[1, , drop = FALSE]`.* »

« *Forgetting to disable the dropping feature is a common cause of failure in general subroutines where an index occasionally, but not usually, has length one.* »

Pourquoi cette phrase est la plus importante de la section. Le bug ne se manifeste pas pendant les tests : tant que la sélection porte sur plusieurs lignes, tout va bien. Le jour où le filtre n'en retient qu'une, le résultat cesse d'être une matrice, et `nrow()` rend `NULL`, `[, 2]` échoue, la fonction plante. **Le code est faux depuis le début et paraissait juste.**

```
m[1, ]           # un VECTEUR de longueur n
m[1, , drop = FALSE]  # une MATRICE 1 x n      <- ce qu'il faut dans une fonction
```

« This rule **still applies to a one-dimensional array**, where **any subsetting will give a vector result** unless `drop = FALSE` is used. »

4.4 Vecteur contre tableau à une dimension

Distinction (§3.4.2). *« Notice that **vectors are distinct from one-dimensional arrays** in that **the latter have `dim` and `dimnames` attributes (both of length one)**. One-dimensional arrays **are not easily obtained from subsetting operations** but they **can be constructed explicitly** and **are returned by `table`**. This is sometimes useful because **the elements of the `dimnames` list may themselves be named**, which is **not the case for the `names` attribute**. »*

C'est l'explication d'un mystère fréquent : le résultat de `table()` **n'est pas un vecteur nommé ordinaire**, il a un `dim` de longueur 1 — d'où un affichage avec un en-tête que `names()` seul ne saurait produire. Voir fiche 305.

« Some operations such as `m[FALSE, ]` result in structures in which **a dimension has zero extent**. *R* generally tries to handle these structures sensibly. »

Énoncé. Une fonction sélectionne les lignes d'une matrice vérifiant une condition, puis calcule la moyenne de chaque colonne.

```
moyennes <- function(m, garde) {  
  sous <- m[garde, ]  
  colMeans(sous)  
}
```

Elle fonctionne pendant des mois, puis échoue. Pourquoi, et comment la corriger ?

Étape 1 — reproduire. Avec une matrice 5×3 et `garde` retenant **trois** lignes, `sous` est une matrice 3×3 et `colMeans` rend trois moyennes. Tout va bien.

Étape 2 — le cas qui casse. Le jour où `garde` ne retient **qu'une seule ligne**, la règle du §3.4.2 s'applique : « if an indexing operation causes the result to have **one of its extents of length one** ... the corresponding dimension is **generally dropped** ». `sous` n'est plus une matrice 1×3 : c'est **un vecteur de longueur 3**.

Étape 3 — le symptôme. `colMeans` exige un objet ayant des dimensions. Sur un vecteur, il lève une erreur du type « 'x' must be an array of at least two dimensions ». L'erreur ne mentionne **ni l'indexation, ni `drop`** — rien ne pointe vers la vraie cause.

Étape 4 — pourquoi le bug a mis des mois à sortir. Le manuel le dit exactement : « *a common cause of failure in general subroutines where an index **occasionally, but not usually, has length one*** ». Le cas limite est **rare**, donc absent des jeux d'essai.

Étape 5 — la correction.

```
moyennes <- function(m, garde) {  
  sous <- m[garde, , drop = FALSE]  
  colMeans(sous)  
}
```

Étape 6 — vérifier qu'on n'a pas décalé les indices. « Notice that this is **an additional argument to the [function and doesn't add to the index count** ». `m[garde, , drop = FALSE]` a toujours **deux** indices : `garde` et l'indice de colonne vide. La virgule avant `drop` reste indispensable.

Étape 7 — généraliser. **Dans une fonction, tout [à deux indices sur une matrice devrait porter `drop = FALSE`**, sauf intention contraire explicite. En interactif, la suppression est commode ; en bibliothèque, c'est une bombe à retardement.

Étape 8 — le cas zéro. Si `garde` ne retient **aucune** ligne, `drop = FALSE` rend une matrice **0 × 3** — « *R generally tries to handle these structures sensibly* » — et `colMeans` rend trois `NaN`. C'est un résultat exploitable, là où la version sans `drop` aurait rendu un vecteur vide.

● Concept 5 — Indexer les autres structures

Règle (§3.4.3). *« The operator `[` is a generic function which allows class methods to be added, and the `$` and `[[` operators likewise. Thus, it is possible to have **user-defined indexing operations for any structure**. Such a function, say `[.foo`, is called with a set of arguments of which **the first is the structure being indexed** and the rest are the indices. »*

« In the case of `$`, **the index argument is of mode "symbol"** even when using the `x$"abc"` form. »

« **It is important to be aware that class methods do not necessarily behave in the same way as the basic methods, for example with respect to partial matching.** »

C'est un avertissement de portée générale : ce que vous venez d'apprendre décrit **les méthodes de base**. Un objet d'une classe particulière peut légitimement indexer autrement. Voir fiche 311.

5.1 Le data frame — le cas le plus important

Règle (§3.4.3). « *The most important example of a class method for `[]` is that used for data frames. ... In broad terms, if two indices are supplied (even if one is empty) it creates matrix-like indexing for a structure that is basically a list of vectors of the same length. If a single index is supplied, it is interpreted as indexing the list of columns — in that case the `drop` argument is ignored, with a warning.* »

Écriture	Interprétation
<code>df[i, j]</code>	indexation matricielle : lignes et colonnes
<code>df[i,]</code>	des lignes (deux indices, le second vide)
<code>df[, j]</code>	des colonnes (deux indices, le premier vide)
<code>df[j]</code>	des colonnes — un seul indice indexe la liste des colonnes

`df[2]` et `df[2, ]` sont deux choses différentes : la **deuxième colonne** contre la **deuxième ligne**. La virgule seule fait la différence, et les deux écritures sont valides. Voir `[.data.frame]` et la fiche 306.

5.2 Les environnements

« *The basic operators `$` and `[]` can be applied to environments. Only character indices are allowed and no partial matching is done.* » (§3.4.3)

Deux restrictions par rapport aux listes — pas d'index numérique (un environnement **n'est pas ordonné**), pas d'appariement partiel. Voir fiche 310.

● Concept 6 — Assigner dans un sous-ensemble

DÉFINITION (§2.7).

« An indexed expression can also appear on **the receiving end of an assignment**, in which case the assignment operation is **performed only on those elements** of the vector. »

```
x[is.na(x)] <- 0      # remplace les manquants par des zeros  
y[y < 0] <- -y[y < 0] # « has the same effect as y <- abs(y) »
```

La restriction du cours (§2.7). *« The expression **must be of the form** `vector[index_vector]` , as **having an arbitrary expression in place of the vector name does not make much sense here.** »*

On peut indexer une expression **pour lire** — `(x+1)[cond]` — mais pas **pour écrire** : `(x+1)[cond] <- 0` n'a pas de sens, puisqu'il n'y a aucun objet à modifier.

CONTRÔLE — VÉRIFIER `Y[Y < 0] <- -Y[Y < 0]`.

Pour `y = c(-3, 2, -1)` : la condition vaut `TRUE FALSE TRUE` , le membre droit `-y[y < 0]` vaut `c(3, 1)` , et ces deux valeurs remplacent les positions 1 et 3. Résultat `c(3, 2, 1)` — bien `abs(y)` . **Les longueurs concordent : 2 positions sélectionnées, 2 valeurs fournies.** Si elles ne concordait pas, le recyclage s'appliquerait (fiche 301).

● Concept 7 — Ce qui se passe vraiment : le mécanisme ``*tmp*``

DÉFINITION (§3.4.4).

« Assignment to subsets of a structure is **a special case of a general mechanism for complex assignment** : »

```
x[3:5] <- 13:15
```

« The result of this command is **as if the following had been executed** : »

```
`*tmp*` <- x
x <- "[<-"(`*tmp*`, 3:5, value = 13:15)
rm(`*tmp*`)
```

EN CLAIR — CE QUE CELA SIGNIFIE.

`x[3:5] <- 13:15` **n'est pas une modification sur place.** C'est : copier `x` dans un objet temporaire, appeler la fonction ``[<-`` qui rend un **nouvel** objet, réassigner ce nouvel objet à `x`, effacer le temporaire. Toute la sémantique de R découle de là — c'est pourquoi **modifier un objet dans une fonction ne modifie jamais l'original** (fiche 310).

Deux conséquences pratiques données par le manuel.

1. « Note that **the index is first converted to a numeric index and then the elements are replaced sequentially along the numeric index, as if a `for` loop had been used.** » L'ordre de remplacement est donc **déterminé** — ce qui compte si le même indice apparaît deux fois : **le dernier gagne**.
2. « **Any existing variable called ``*tmp*`` will be overwritten and deleted, and this variable name should not be used in code.** »

7.1 Les fonctions de remplacement — la règle générale

FORMULE (§3.4.4).

« **The same mechanism can be applied to functions other than `[]`. The replacement function has the same name with `<-` pasted on. Its last argument, which must be called `value`, is the new value to be assigned.** »

```
names(x) <- c("a", "b")
```

est équivalent à

```
`*tmp*` <- x
x <- "names<-"(`*tmp*`, value = c("a", "b"))
```

```
rm(`*tmp*`)
```

C'est l'explication de toute une famille de constructions qui ressemblent à des affectations de propriété et n'en sont pas :

Écriture	Fonction réellement appelée
<code>names(x) <- v</code>	<code>`names<-`(x, value = v)</code>
<code>dim(x) <- v</code>	<code>`dim<-`(x, value = v)</code>
<code>levels(f) <- v</code>	<code>`levels<-`(f, value = v)</code>
<code>class(x) <- v</code>	<code>`class<-`(x, value = v)</code>
<code>x[i] <- v</code>	<code>`[<-`(x, i, value = v)</code>
<code>is.na(x) <- i</code>	<code>`is.na<-`(x, value = i)</code>

Ce qui explique aussi le passage de la fiche 300 : `assign("names(x)", nm)` **ne fait pas** ce que fait `names(x) <- nm`, puisque `assign` prend le texte pour un nom d'objet et **ne connaît pas les fonctions de remplacement**.

Rappel du §3.1.3. *« A special type of function calls can appear on the left hand side of the assignment operator, as in `class(x) <- "foo"` . **What this construction really does is to call the function `class<-` with the original object and the right hand side. ... (At least conceptually, this is what happens. Some additional effort is made to avoid unnecessary data duplication.)**

»* — la copie décrite est **le modèle sémantique**, pas nécessairement ce que fait la machine.

Comment reconnaître le type d'exercice

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« garder les éléments qui vérifient... »	index logique , <code>x[cond]</code> — et la longueur change
« retirer les cinq premiers »	index négatif , <code>x[-(1:5)]</code> — parenthèses obligatoires
« dans cet ordre-là »	index entier positif ou caractère : le résultat suit l'ordre de l'index
« accéder par nom »	index caractère , via l'attribut <code>names</code>
le nom de la colonne est dans une variable	<code>x[[nom]]</code> , jamais <code>\$</code>
« pourquoi ai-je <code>NULL</code> ? »	<code>\$</code> avec un nom qui n'existe pas — ou un nom calculé
« je lis la bonne valeur mais l'écriture crée un doublon »	appariement partiel : actif en lecture, inactif en écriture
« mon résultat a un <code>NA</code> en trop »	index positif hors bornes , ou index logique plus long que le vecteur
« je crois avoir retiré un élément, rien n'a bougé »	index négatif hors bornes — silencieusement ignoré
« le résultat a une longueur inattendue »	des zéros dans l'index, ou une troncature d'index non entier
« j'indexe par un facteur »	c'est <code>x[as.integer(i)]</code> — les niveaux ne sont pas utilisés
« <code>nrow()</code> rend <code>NULL</code> dans ma fonction »	drop : la dimension de longueur 1 a été supprimée
« <code>m[1]</code> ne me rend pas la première ligne »	un seul indice ignore <code>dim</code> : c'est <code>c(m)[1]</code>
« extraire des cases dispersées sans boucle »	index matriciel : une ligne = un jeu de coordonnées
« <code>df[2]</code> ou <code>df[2,]</code> ? »	colonne contre ligne — <code>[.data.frame]</code>
« pourquoi mon objet n'est pas modifié dans la fonction ? »	le mécanisme <code>`*tmp*`</code> : l'assignation recrée l'objet

Comment résoudre ce type d'exercice

Protocole « prévoir le résultat d'un `x[i]` » — 5 étapes.

1. **Identifier le type de `i`** : logique, entier positif, entier négatif, caractère, facteur, vide, `NULL` , `NA` .
Sept comportements différents.
2. **Vérifier les signes** : dans un index entier, tous les éléments doivent avoir **le même signe**.
3. **Déterminer la longueur du résultat** — c'est là que sont les surprises : logique → longueur de `x` (après recyclage) ; entier → **longueur de l'index** ; négatif → `length(x)` moins les valides ; `x[0]` et `x[NULL]` → **0**.
4. **Traiter les cas de bord** : hors bornes positif → `NA` ; hors bornes négatif → **ignoré** ; zéros → **ignorés** ; non entiers → **tronqués vers zéro**.
5. **Vérifier `length()`** sur le résultat, systématiquement.

Protocole « indexer proprement dans une fonction » — 4 étapes.

1. Nom de composante venant d'une variable → `[[` , jamais `$` .
2. Deux indices sur une matrice ou un data frame → `drop = FALSE` par défaut.
3. Appariement partiel non souhaité → `x[[nom, exact = TRUE]]` , ou `[` qui exige toujours l'exactitude.
4. Après le sous-ensemble, **vérifier la classe et les dimensions** avant de les utiliser.

Protocole « modifier une partie d'un objet » — 4 étapes.

1. Écrire l'assignation sous la forme `objet[index] <- valeurs` — jamais une expression à gauche.
2. **Compter** : combien de positions sélectionnées, combien de valeurs fournies. Longueurs égales, ou un multiple exact.
3. Se souvenir que le remplacement est **séquentiel le long de l'index numérique** : un indice répété → **la dernière valeur gagne**.
4. Se souvenir qu'**un nouvel objet est créé** : à l'intérieur d'une fonction, seule la copie locale change.

● Common mistakes

Erreur	Correction
Utiliser <code>\$</code> avec un nom stocké dans une variable	<code>\$</code> n'évalue pas son index → <code>x[[nom]]</code>
S'attendre à une erreur de <code>x\$inexistent</code>	il rend <code>NULL</code> , silencieusement
Croire que <code>l[1]</code> et <code>l[[1]]</code> sont pareils	<code>[</code> rend une liste , <code>[[</code> rend l'élément
Croire que <code>l[[c(1,2)]]</code> prend deux éléments	c'est une descente en profondeur : élément 2 de l'élément 1
Écrire <code>x[-1:5]</code> pour « tout sauf 1 à 5 »	signes mélangés → erreur ; écrire <code>x[-(1:5)]</code>
Croire un index positif hors bornes sans effet	il produit un <code>NA</code>
Croire un index négatif hors bornes signalé	il est silencieusement ignoré (depuis R 2.6.0)
Compter <code>x[c(1, 0, 3)]</code> pour 3 éléments	les zéros sont ignorés → 2 éléments
Croire <code>x[2.9]</code> invalide	il vaut <code>x[2]</code> — troncature vers zéro , sans avertissement
Indexer par un facteur en croyant utiliser ses niveaux	c'est <code>x[as.integer(i)]</code> → utiliser <code>x[as.character(i)]</code>
Croire <code>x[]</code> équivalent à <code>x</code>	il laisse tomber les attributs « non pertinents »
Croire <code>x[NA]</code> et <code>x[c(1,NA)]</code> de même longueur	règles logiques contre règles entières
Compter sur l'appariement partiel en écriture	« <i>only used when extracting, not when replacing</i> » → un élément est créé
Croire <code>[</code> capable d'appariement partiel	<code>[</code> exige toujours l'exactitude
Utiliser <code>""</code> comme nom	« <i>indicates "no name" and matches no element</i> »
Croire <code>m[1]</code> égal à <code>m[1,]</code>	un seul indice ignore <code>dim</code> : c'est <code>c(m)[1]</code>
Oublier <code>drop = FALSE</code> dans une fonction	la dimension de longueur 1 disparaît — panne différée
Croire que <code>drop</code> décale les indices	« <i>doesn't add to the index count</i> » : garder la virgule
Mettre des indices négatifs dans une matrice d'index	interdit

Erreur	Correction
Croire les lignes à zéro d'une matrice d'index sélectionnantes	elles sont ignorées ; celles à <code>NA</code> produisent <code>NA</code>
Confondre <code>df[2]</code> et <code>df[2,]</code>	colonne contre ligne
Passer <code>drop</code> à un <code>df[j]</code> à un seul indice	ignoré, avec un avertissement
Indexer un environnement par un entier	seuls les index caractères sont permis, sans appariement partiel
Croire <code>x[i] <- v</code> modifier sur place	c'est <code>`[<`(x, i, value = v)</code> — un nouvel objet
Utiliser une variable nommée <code>`*tmp*`</code>	elle est écrasée et détruite par toute assignation complexe
Écrire une fonction de remplacement sans <code>value</code>	le dernier argument doit s'appeler <code>value</code>

Ultimate Review

Trois opérateurs. `[]` prend un **sous-ensemble** (accepte un vecteur d'indices, conserve le type, **jamais d'appariement partiel**) · `[[` extrait **un seul élément** (perd `names` / `dimnames`, appariement partiel possible) · `$` extrait par **nom littéral** (index **non calculable**, appariement partiel **silencieux**, **erreur** sur un objet non récursif). Pour une liste : `l[1]` est une **liste**, `l[[1]]` est **l'élément**. `l[[c(1,2)]]` **descend en profondeur**.

Les quatre index du cours (§2.7). Logique — recyclé si court, `x` prolongé par des `NA` s'il est long ; sélectionne les `TRUE`. **Entier positif** — valeurs dans $\{1, \dots, n\}$, **le résultat a la longueur de l'index**, l'ordre est celui de l'index. **Entier négatif** — **exclusion** ; parenthéser : `x[-(1:5)]`. **Caractère** — via `names`.

Les quatre cas de bord (§3.4.1). Facteur = `x[as.integer(i)]`, **les niveaux ne servent pas. Vide** `x[]` rend `x` **en laissant tomber les attributs non pertinents** (garde `names`, et `dim` / `dimnames` en multi-dimensionnel). `NULL` = `integer(0)`. `NA` → résultat `NA` ; `x[NA]` **a la longueur de** `x` (règles logiques), `x[c(1,NA)]` **a la longueur 2** (règles entières).

Les règles silencieuses. Positif **hors bornes** → `NA` · négatif hors bornes **ignoré** (depuis **R 2.6.0**) · `x[0]` **vide**, zéros mélangés **ignorés** · non entiers **tronqués vers zéro** · **tous les éléments d'un index entier ont le même signe** · `[]` **sous-ensemble aussi les** `names`.

Appariement partiel. `[]` **jamais** · `[[` selon `exact` : `NA` (défaut) autorisé **avec avertissement**, `TRUE` interdit, `FALSE` autorisé en silence · `$` **oui, en silence** · **seulement en extraction, jamais en remplacement** · `""` signifie « **pas de nom** » et **n'apparie rien**.

Matrices (§3.4.2). Un seul indice → `dim` / `dimnames` **ignorés**, résultat `c(m)[i]` : `m[1]` ≠ `m[1, ]` ≠ `m[, 1]` · **index matriciel** : autant de colonnes que de dimensions, **une ligne = un jeu de coordonnées**, résultat de longueur `nrow(index)` ; **pas d'indices négatifs**, lignes à **zéro ignorées**, lignes à `NA` → `NA` · **drop** : toute extension de longueur 1 est **supprimée** → `m[1, , drop = FALSE]`, argument **qui ne compte pas dans les indices** ; « *a common cause of failure in general subroutines* » · **un tableau 1-D n'est pas un vecteur** (il a `dim` et `dimnames` de longueur 1) et `table()` en rend un · `m[FALSE, ]` donne une extension nulle.

Autres structures (§3.4.3). `[`, `[[`, `$` sont **génériques** : `[.foo]` reçoit l'**objet d'abord**, puis les indices ; pour `$`, l'index est **de mode "symbol"**, même écrit `x$"abc"` · **les méthodes de classe ne se comportent pas nécessairement comme les méthodes de base** · **data frame** : deux indices → **matriciel** ; un seul indice → **les colonnes**, et **drop** est ignoré avec un avertissement · **environnements** : **index caractères seulement**, aucun appariement partiel.

Assignation (§2.7 et §3.4.4). `x[cond] <- v` n'agit **que sur les éléments sélectionnés** · **le membre gauche doit être** `vecteur[index]`, pas une expression quelconque · le mécanisme réel est

```
`*tmp*` <- x ; x <- `[<-(`*tmp*`, i, value = v) ; rm(`*tmp*`)
```

· l'index est **converti en index numérique**, puis les éléments **remplacés séquentiellement, comme par une boucle for** · **ne jamais nommer une variable** ``*tmp*`` · **fonction de remplacement** = le nom plus `<-`, dont le **dernier argument s'appelle** `value` : `names<-`, `dim<-`, `class<-`, `levels<-`, `is.na<-`.

Active Recall

« Such index vectors can be **any of four distinct types**. » (§2.7)

1. **Un vecteur logique** — « the index vector is **recycled to the same length** as the vector ... values corresponding to `TRUE` are selected ». Sert à **filtrer** : `x[!is.na(x)]`.
2. **Un vecteur d'entiers positifs** — valeurs dans $\{1, \dots, n\}$; « The index vector can be **of any length** and the result is of **the same length as the index vector** ». Sert à **choisir et réordonner**.
3. **Un vecteur d'entiers négatifs** — « specifies the values **to be excluded** rather than included » : `x[-(1:5)]`.
4. **Un vecteur de chaînes** — « only applies where an object **has a names attribute** ». Sert à accéder **par nom**.

La R Language Definition en ajoute quatre : **facteur**, **vide**, **NULL** , **NA** .

« The form using `$` ... allows **only a literal character string or a symbol** as the index. That is, **the index is not computable** : for cases where you need to **evaluate an expression** to find the index, use `x[[expr]]` . » (§3.4)

`donnees$colonne` cherche donc une composante **littéralement nommée** `colonne` . Comme elle n'existe pas, R rend **NULL** — **sans erreur**. Le symptôme n'apparaît qu'en aval, sous une forme sans rapport.

La bonne écriture : `donnees[[colonne]]` .

« The indexing `i` should generally have the same length as `x` . **If it is shorter, then its elements will be recycled ... If it is longer, then `x` is conceptually extended with `NA` s.** » (§3.4.1)

- **Plus court** : recyclage — d'où l'idiome `x[c(TRUE, FALSE)]` , qui prend **un élément sur deux**.
- **Plus long** : `x` est prolongé par des `NA` , qui apparaissent **à la fin du résultat**.

Le second cas est presque toujours un bug : deux vecteurs qu'on croyait alignés ne le sont pas.

- `x[7]` → `NA` . « If `i` is positive and **exceeds** `Length(x)` then the corresponding selection is `NA` ».
- `x[-7]` → `x` **en entier**. « **Negative out of bounds values for `i` are silently disregarded** since R version **2.6.0**, *S* compatibly, as they mean to **drop non-existing elements** and that is **an empty operation ("no-op")** ».

La logique de l'asymétrie : demander l'élément 7 d'un vecteur de 5 est une **question sans réponse** → `NA` . Demander de **retirer** l'élément 7 est une **instruction sans effet** → rien. Le premier signale, le second se tait.

- `x[0]` : « a special case is the zero index, which has **null effects** : `x[0]` is **an empty vector** » → longueur **0**.
- `x[c(1, 0, 3)]` : « including zeros among positive or negative indices has the same effect **as if they were omitted** » → équivaut à `x[c(1, 3)]` , longueur **2**.
- `x[2.9]` : « **Other numeric**. Non-integer values are **converted to integer (by truncation towards zero)** » → `x[2]` , sans avertissement.
- `x[NULL]` : « **NULL**. This is treated as if it were `integer(0)` » → vecteur **vide**.

*« **Factor**. The result is identical to `x[as.integer(i)]` . **The factor levels are never used**. If so desired, use `x[as.character(i)]` or a similar construction. »* (§3.4.1)

Un facteur **s'affiche avec ses étiquettes** mais **vaut ses codes entiers** (fiche 304). Indexer par un facteur indexe donc **par des positions 1, 2, 3...** que rien à l'écran ne laisse deviner : le résultat a la **bonne longueur**, et il est **faux**.

Pour indexer par les étiquettes, il faut `as.character()` .

*« there are **different modes of NA** — the literal constant is of mode "logical" , but it is **frequently automatically coerced** to other types. One effect of this is that `x[NA]` **has the length of** `x` , but `x[c(1, NA)]` **has length 2**. That is because **the rules for logical indices apply in the former case, but those for integer indices in the latter**. »* (§3.4.1)

- `NA` **seul** est **logique** → index logique **recyclé** sur toute la longueur, résultat de longueur `length(x)` , tout en `NA` .
- `c(1, NA)` **coerce** le `NA` en **entier** → index entier de longueur 2, résultat de longueur 2.

C'est la **coercition dans** `c()` qui change la règle.

« For `[[` and `$` **partial matching is used if exact matching fails ... Note that `[` always requires an exact match.** » Et pour `[[` , l'argument `exact` : défaut `NA` (autorisé avec avertissement), `TRUE` (interdit), `FALSE` (autorisé sans avertissement). (§3.4.1)

La restriction critique : « *partial matching is only used when extracting and not when replacing* ».

Conséquence : `l$val` **lit** `l$valeur` , mais `l$val <- 3` **crée un nouvel élément nommé** `val` à côté de `valeur` . On croit modifier, on ajoute — et la lecture suivante, `l$val` , trouvera désormais l'élément exact et non plus l'ancien.

Enfin : `""` « *indicates "no name" and matches no element (not even those without a name)* ».

*« It is however also possible to use **a single index**, in which case **the** `dim` **and** `dimnames` **attributes are disregarded** and the result is effectively that of `c(m)[i]` . **Notice that** `m[1]` **is usually very different from** `m[1, ]` **or** `m[, 1]` . »* (§3.4.2)

- `m[1]` → **le premier élément du vecteur sous-jacent** (colonne par colonne, R stockant en ordre colonne-majeur).
- `m[1, ]` → **la première ligne**.
- `m[, 1]` → **la première colonne**.

Pourquoi : une matrice **est un vecteur muni d'un** `dim` (§2.8). Avec un seul indice, `dim` est **mis de côté** et l'on retombe sur le vecteur.

Par un **index matriciel** (§3.4.2) : « *the number of columns of the matrix should match the number of dimensions of the structure, and the result will be a vector with length as the number of rows of the matrix* ».

```
m <- matrix(1:4, 2)
i <- matrix(c(1, 1, 2, 2), 2, byrow = TRUE)
m[i]
# [1] 1 4
```

Chaque **ligne** de `i` est **un jeu de coordonnées**.

« Indexing matrices **may not contain negative indices**. `NA` and zero values are allowed : **rows containing a zero are ignored**, whereas **rows containing an `NA` produce an `NA` in the result**. »

*« If an indexing operation causes the result to have **one of its extents of length one ... the corresponding dimension is generally dropped from the result**. If a single-dimensional structure results, **a vector is obtained**. ... can be turned off by adding `drop = FALSE` . »* (§3.4.2)

« **Forgetting to disable the dropping feature is a common cause of failure in general subroutines where an index occasionally, but not usually, has length one**. »

Pourquoi c'est insidieux : tant que la sélection porte sur plusieurs lignes, tout marche. Le jour où elle n'en retient qu'une, le résultat **cesse d'être une matrice** — `nrow()` rend `NULL` , `[ , 2]` échoue — et l'erreur ne mentionne **ni l'indexation ni `drop`** .

L'écriture correcte : `m[1, , drop = FALSE]` , en notant que « *this is an additional argument to the `[` function and **doesn't add to the index count*** » — la virgule reste.

*« **Vectors are distinct from one-dimensional arrays** in that **the latter have `dim` and `dimnames` attributes (both of length one)**. One-dimensional arrays **are not easily obtained from subsetting operations** but they can be **constructed explicitly** and **are returned by `table`** . This is sometimes useful because **the elements of the `dimnames` list may themselves be named**, which is **not the case for the `names` attribute**. »* (§3.4.2)

C'est pourquoi le résultat de `table()` s'affiche avec **un en-tête nommé** : le nom de la variable tabulée est **le nom de l'élément de `dimnames`** , chose qu'un simple `names` ne peut pas porter.

« **if two indices are supplied (even if one is empty) it creates matrix-like indexing ... If a single index is supplied, it is interpreted as indexing the list of columns** — in that case the `drop` argument is ignored, with a warning. » (§3.4.3)

- `df[2]` → la **deuxième colonne** (un data frame d'une colonne).

- `df[2, ]` → la **deuxième ligne**.

Une virgule sépare les deux, et **les deux écritures sont valides** : rien ne signale l'erreur. Un data frame est « *basically a list of vectors of the same length* », et l'indexation à un seul indice le traite **comme la liste qu'il est**.

« *The result of this command is **as if the following had been executed*** » (§3.4.4) :

```
`*tmp*` <- x
x <- "[<-"(`*tmp*`, 3:5, value = 13:15)
rm(`*tmp*`)
```

Ce n'est **pas** une modification sur place : c'est **une copie, un appel de fonction** qui rend un **nouvel objet, une réassignation**, puis **l'effacement du temporaire**. C'est de là que vient le fait qu'une fonction ne peut pas modifier son argument (fiche 310).

Deux précisions : « *the index is **first converted to a numeric index** and then the elements are **replaced sequentially along the numeric index, as if a for loop had been used*** » — donc un indice **répété** voit **la dernière valeur l'emporter**. Et : « *any existing variable called ``*tmp*`` will be overwritten and deleted, and this variable name **should not be used in code*** ».

« *The **replacement function** has the same name with `<-` pasted on. Its last argument, which must be called `value`, is the new value to be assigned.* » (§3.4.4)

Ainsi `names(x) <- c("a","b")` **est** l'appel ``names<-`(x, value = c("a","b"))` suivi d'une réassignation à `x`.

Toute une famille suit cette règle : ``dim<-``, ``class<-``, ``levels<-``, ``is.na<-``, ``[<-``, ``[[<-``, ``$<-``.

C'est aussi ce qui explique la mise en garde de la fiche 300 : `assign("names(x)", nm)` **n'est pas** `names(x) <- nm`, puisque `assign` prend la chaîne pour **un nom d'objet** et ignore les fonctions de remplacement.

« The operator `[]` **is a generic function which allows class methods to be added**, and the `$` and `[]` operators likewise. ... **It is important to be aware that class methods do not necessarily behave in the same way as the basic methods**, for example **with respect to partial matching**. » (§3.4.3)

Tout ce qui précède décrit **les méthodes de base**. Une classe peut définir `{}.foo` — appelé « *with a set of arguments of which **the first is the structure being indexed** and the rest are the indices* » — et lui donner d'autres règles. Le data frame en est l'exemple le plus courant : `df[j]` à un seul indice **n'a rien du** `m[i]` **d'une matrice**.

Flashcards

Question	Réponse
Les trois opérateurs d'indexation ?	<code>[</code> , <code>[[</code> , <code>\$</code>
Ce que rend <code>[</code> ?	Un sous-ensemble, du même type que l'objet
Ce que rend <code>[[</code> ?	Un seul élément , dépouillé de <code>names</code> / <code>dimnames</code>
<code>l[1]</code> contre <code>l[[1]]</code> pour une liste ?	Une liste contre l'élément
Que fait <code>l[[c(1,2)]]</code> ?	Descend en profondeur : élément 2 de l'élément 1
<code>\$</code> accepte quoi comme index ?	Une chaîne littérale ou un symbole — non calculable
Nom de composante dans une variable ?	<code>x[[nom]]</code> , jamais <code>\$</code>
<code>\$</code> sur un objet non récursif ?	Une erreur
Que rend <code>x\$inexistant</code> ?	<code>NULL</code> , silencieusement
Les quatre types d'index du cours ?	Logique · entier positif · entier négatif · caractère
Index logique trop court ?	Recyclé
Index logique trop long ?	<code>x</code> est prolongé par des <code>NA</code>
Longueur du résultat d'un index entier positif ?	Celle de l'index
L'ordre du résultat ?	Celui de l'index
Que fait un index négatif ?	Il exclut
Pourquoi <code>x[-(1:5)]</code> et pas <code>x[-1:5]</code> ?	<code>:</code> prioritaire → signes mélangés , interdits
Index entier positif hors bornes ?	<code>NA</code>
Index négatif hors bornes ?	Silencieusement ignoré (depuis R 2.6.0)
Que vaut <code>x[0]</code> ?	Un vecteur vide
Un zéro parmi d'autres indices ?	Ignoré

Question	Réponse
Que vaut <code>x[2.9]</code> ?	<code>x[2]</code> — troncature vers zéro
Que vaut <code>x[NULL]</code> ?	Comme <code>integer(0)</code> → vide
Longueur de <code>x[NA]</code> ?	<code>length(x)</code> — règles logiques
Longueur de <code>x[c(1, NA)]</code> ?	2 — règles entières
Indexer par un facteur ?	<code>x[as.integer(i)]</code> — niveaux non utilisés
Comment indexer par les étiquettes d'un facteur ?	<code>x[as.character(i)]</code>
Que fait <code>x[]</code> ?	Rend <code>x</code> en laissant tomber les attributs non pertinents
Quels attributs <code>x[]</code> conserve-t-il ?	<code>names</code> , et <code>dim</code> / <code>dimnames</code> en multi-dimensionnel
Signes dans un index entier ?	Tous les mêmes
<code>[]</code> touche-t-il aux noms ?	Oui — il sous-ensemble aussi <code>names</code>
<code>[]</code> fait-il de l'appariement partiel ?	Jamais
Défaut de <code>exact</code> pour <code>[]</code> ?	<code>NA</code> : partiel autorisé, avec avertissement
<code>exact = TRUE</code> ?	Interdit l'appariement partiel
<code>exact = FALSE</code> ?	L' autorise sans avertissement
<code>\$</code> et l'appariement partiel ?	Oui, en silence
L'appariement partiel en écriture ?	Jamais — un élément est créé
Que signifie le nom <code>""</code> ?	« pas de nom » — n'apparie aucun élément
<code>m[1]</code> contre <code>m[1,]</code> ?	Élément du vecteur sous-jacent contre première ligne
Pourquoi ?	Un seul indice ignore <code>dim</code> : c'est <code>c(m)[1]</code>
Index matriciel : combien de colonnes ?	Autant que de dimensions
Longueur du résultat ?	Le nombre de lignes de la matrice d'index

Question	Réponse
Que représente une ligne de l'index ?	Un jeu de coordonnées
Indices négatifs dans une matrice d'index ?	Interdits
Ligne contenant un zéro ?	Ignorée
Ligne contenant un <code>NA</code> ?	Produit un <code>NA</code>
Qu'est-ce que la suppression de dimension ?	Toute extension de longueur 1 est retirée
Comment l'empêcher ?	<code>drop = FALSE</code>
<code>drop</code> compte-t-il comme un indice ?	Non — garder la virgule
Première ligne comme matrice 1 × n ?	<code>m[1, , drop = FALSE]</code>
Pourquoi le manuel parle-t-il d'une cause fréquente d'échec ?	Le cas « une seule ligne » est rare , donc non testé
Vecteur contre tableau 1-D ?	Le second a <code>dim</code> et <code>dimnames</code> (de longueur 1)
Qui rend un tableau 1-D ?	<code>table()</code>
Avantage de <code>dimnames</code> sur <code>names</code> ?	Ses éléments peuvent eux-mêmes être nommés
Que donne <code>m[FALSE,]</code> ?	Une structure à dimension d'extension nulle
<code>[</code> est-il générique ?	Oui , ainsi que <code>\$</code> et <code>[[</code>
Signature de <code>[.foo</code> ?	L'objet d'abord , puis les indices
Mode de l'index de <code>\$</code> ?	<code>"symbol"</code> , même écrit <code>x\$"abc"</code>
<code>df[i, j]</code> ?	Indexation matricielle
<code>df[j]</code> ?	Les colonnes — la liste de colonnes
<code>drop</code> sur un <code>df</code> à un seul indice ?	Ignoré, avec un avertissement
Indexer un environnement ?	<code>\$</code> et <code>[[</code> , chaînes seulement , aucun appariement partiel
Forme obligatoire du membre gauche d'une assignation indexée ?	<code>vecteur[index]</code>

Question	Réponse
Que fait <code>x[is.na(x)] <- 0</code> ?	Remplace les manquants par des zéros
Équivalent de <code>y[y < 0] <- -y[y < 0]</code> ?	<code>y <- abs(y)</code>
Le mécanisme réel de <code>x[i] <- v</code> ?	<code>`*tmp*`</code> → <code>`[<-`</code> → réassignation → <code>rm</code>
L'ordre de remplacement ?	Séquentiel , comme une boucle <code>for</code>
Un indice répété en assignation ?	La dernière valeur gagne
Nom de variable à ne jamais utiliser ?	<code>`*tmp*`</code>
Comment se nomme une fonction de remplacement ?	Le nom plus <code><-</code>
Comment doit s'appeler son dernier argument ?	<code>value</code>
Cinq fonctions de remplacement usuelles ?	<code>names<-</code> · <code>dim<-</code> · <code>class<-</code> · <code>levels<-</code> · <code>is.na<-</code>
Pourquoi <code>assign("names(x)", nm)</code> échoue-t-il ?	<code>assign</code> ignore les fonctions de remplacement